

AI System Design – A Complete Guide (2026)

A Complete Guide

6 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:

<https://snehaliteng.github.io/tutorials/ai-system-design/>

1. Introduction

Why AI System Design Matters

Building an AI model that works in a Jupyter notebook is one thing. Deploying it to production — where it must serve millions of requests with low latency, high availability, and continuous adaptation — is an entirely different challenge. AI system design bridges the gap between ML experimentation and production-grade engineering.

In 2026, AI systems are no longer just about training a model and serving predictions. They involve multi-agent orchestrators, real-time data streams, vector databases, semantic caching, compliance frameworks, and continuous feedback loops.

2026 Trends Shaping AI Systems

Trend	Impact on System Design
Multi-Agent Workflows	Systems decompose tasks across specialized agents, requiring orchestration, inter-agent communication, and shared memory.
MCP (Model Context Protocol)	Standardizes how AI systems connect to external tools and data sources, replacing custom integrations.
LLMOps & Prompt Engineering	Prompt versioning, A/B testing of prompts, guardrails, and cost tracking become first-class concerns.
Real-Time ML	Sub-100ms inference for fraud detection, recommendations, and autonomous decision-making.
AI TRiSM	Trust, Risk, Security, and Management of AI systems — compliance-driven design with explainability.
Small & Specialized Models	Distilled models, LoRA adapters, and domain-specific models replace one-size-fits-all LLMs for cost and latency.

What This Guide Covers

- **Part I:** Foundations — problem space, objectives, architecture, data flow, key components
- **Part II:** Agentic AI & Pipelines — multi-agent systems, batch vs real-time, scalability, caching, indexing, inference
- **Part III:** Operations & Security — data freshness, deployment, fault tolerance, monitoring, privacy, security
- **Part IV:** Case Study, Interview Prep & Beyond — end-to-end recommendation engine, trade-offs, interview questions, resources

Exercise: Think of an AI-powered application you use daily (e.g., ChatGPT, Google Search, Netflix recommendations). Identify at least three system-level components it must have beyond the ML model itself (e.g., user database, request queue, caching layer, monitoring).

2. Problem Space

Defining the Scope

The first step in any AI system design is to clearly define the problem. Without this, you risk building a system that solves the wrong thing or fails to meet real-world constraints.

Ask these questions before designing:

- **What is the business problem?** "Fraud detection" vs "reduce chargeback rate by 30%" — the second is measurable.
- **Who are the users?** Internal analysts? External customers? Automated systems?
- **What is the input and output?** Raw transaction data → fraud probability score.
Free-text query → answer with citations.
- **What is the decision boundary?** At what confidence threshold do we act?
- **What are the success criteria?** Accuracy, latency, throughput, uptime?

Clarifying Constraints

Every system operates within constraints. Identify them early:

Constraint Type	Examples
Latency	API must respond within 200ms p99
Throughput	Handle 10,000 requests/second
Cost	Inference budget <\$0.001 per request
Data Governance	Must comply with GDPR — data cannot leave EU region
Hardware	Must run on CPU (no GPU), 4GB RAM
Model Size	Model must fit in 500MB for edge deployment

Functional vs Non-Functional Requirements

Functional Requirements (what the system does)

- Accept transaction data via REST API
- Return fraud probability score (0.0-1.0)
- Log all predictions for audit trail
- Allow admin to view prediction history

Non-Functional Requirements (how the system performs)

- p99 latency < 200ms
- 99.95% uptime
- Handle 5,000 QPS peak
- Encrypt all data at rest and in transit
- Support canary deployments for model rollouts

Key insight: Non-functional requirements drive most architectural decisions. Latency determines caching and model size. Uptime determines redundancy and failover. Cost determines whether you use GPU or CPU, on-prem or cloud.

Avoiding Common Pitfalls

- **Scope creep:** "Let's also add sentiment analysis" — stick to the primary problem.
- **Over-engineering:** Don't design for 100M users if you have 10K.
- **Ignoring data pipelines:** Most AI failures are data failures, not model failures.
- **Starting with the model:** Start with the problem, not the solution.

Exercise: You are asked to design an AI customer support chatbot for an e-commerce platform. Write down: a) 3 functional requirements, b) 3 non-functional requirements, c) 3 constraints, d) 1 measurable success criterion. Be specific with numbers where possible.

3. Core Objectives

Every AI system design balances five core objectives. Understanding their interactions and trade-offs is essential to making good architectural decisions.

1. Accuracy

The model's ability to produce correct predictions. Measured by task-specific metrics: accuracy, precision, recall, F1, RMSE, perplexity, etc.

- **Trade-off:** Higher accuracy often requires larger models, more data, and longer training — increasing cost and latency.
- **Strategies:** Ensemble methods increase accuracy at the cost of serving complexity. Distillation reduces model size while retaining most accuracy.

2. Scalability

The ability to handle growing load (data volume, request rate, model count) without degrading performance.

- **Horizontal scaling:** Add more inference servers behind a load balancer.
- **Vertical scaling:** More powerful GPUs/TPUs for training.
- **Data scalability:** Feature stores, data versioning, and streaming pipelines that handle TB/day.

3. Latency

End-to-end response time from request to prediction. Critical for real-time applications.

Application	Target Latency
Real-time fraud detection	<100ms
Chatbot response	<500ms
Document summarization	<5s (async acceptable)
Batch processing (training)	Hours to days

Latency budget breakdown: Network (5ms), feature fetch (20ms), model inference (50ms), post-processing (10ms), cache lookups (5ms). Budgeting these explicitly helps identify bottlenecks.

4. Adaptability

The system's ability to adapt to changing data distributions, user behavior, and business requirements.

- **Model retraining:** Scheduled (daily/weekly) or triggered (drift detected).
- **Online learning:** Models update incrementally with new data (e.g., bandit algorithms).
- **Feature drift:** Detect when input distributions shift and alert before model quality degrades.
- **Configurability:** Feature flags, model version routing, prompt versioning.

5. Observability

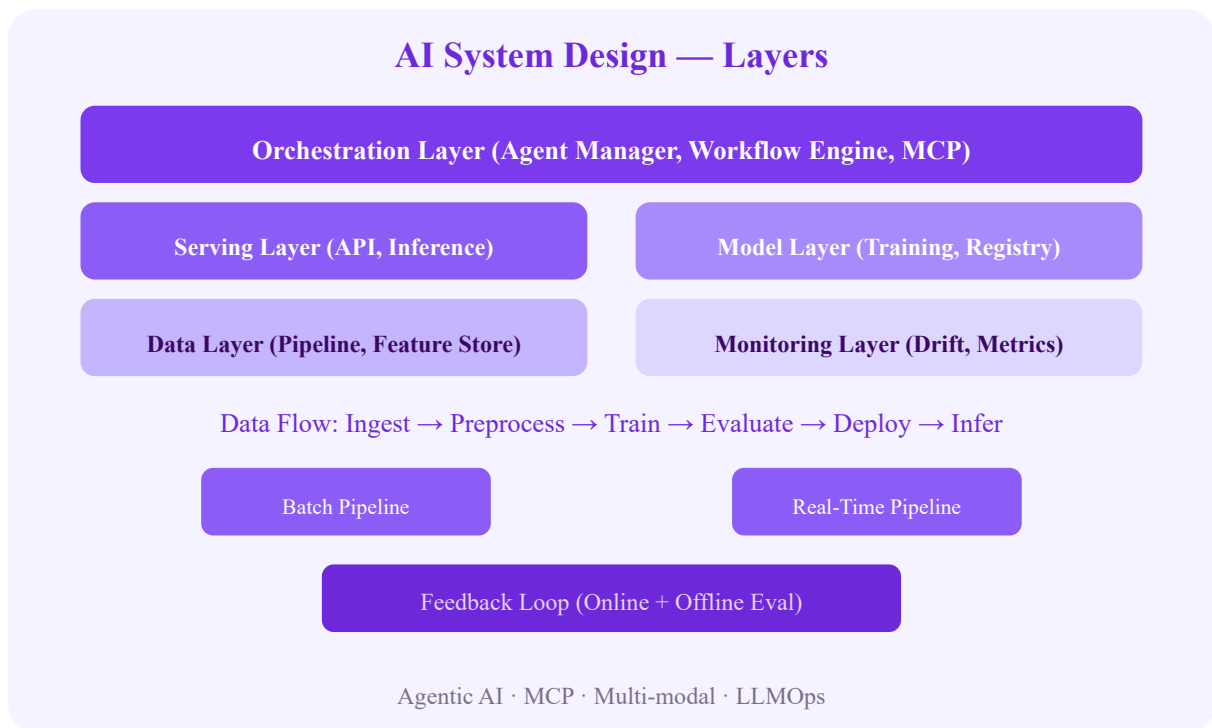
The ability to understand the system's internal state from its external outputs. For AI systems, this means:

- **Model metrics:** Latency histograms, throughput, error rates, prediction distribution.
- **Data drift:** Statistical divergence between training and serving data.
- **Prediction explanations:** SHAP/LIME values for model interpretability.
- **Resource utilization:** GPU/CPU/memory usage, queue depths, cache hit ratios.
- **Alerting:** Automated alerts when metrics breach thresholds.

```
# Observable metrics for an inference service
latency_p50: 45ms
latency_p99: 185ms
throughput: 3,200 req/s
error_rate: 0.02%
cache_hit_ratio: 0.78
model_drift_score: 0.12 (threshold: 0.2 → retrain)
```

Exercise: For a real-time translation service, rank the five objectives (accuracy, scalability, latency, adaptability, observability) by priority and explain your ordering. Describe a scenario where improving one objective would hurt another.

4. High-Level Architecture



A production AI system can be understood as four interconnected layers, with a fifth cross-cutting layer for observability.

Data Layer

Responsible for ingesting, storing, processing, and serving data for both training and inference.

- **Data ingestion:** Batch (Airflow) and streaming (Kafka) pipelines.
- **Feature store:** Centralized repository of features (Feast, Tecton, SageMaker Feature Store).
- **Data lake / warehouse:** Raw and processed data storage (S3, Snowflake, BigQuery).
- **Data versioning:** DVC, LakeFS — track dataset versions for reproducibility.

Model Layer

Manages model training, evaluation, packaging, and lifecycle.

- **Training infrastructure:** Distributed training (PyTorch DDP, DeepSpeed), hyperparameter tuning (Optuna, Ray Tune).
- **Model registry:** Versioned model storage with metadata (MLflow, W&B, Hugging Face Hub).
- **Evaluation:** Offline eval on held-out datasets, online eval via A/B testing.
- **Compression:** Quantization (FP16, INT8), pruning, distillation for deployment.

Serving Layer

Exposes the model to consumers via APIs with low latency and high throughput.

- **Inference server:** NVIDIA Triton, TorchServe, BentoML, vLLM, TGI.
- **API gateway:** Authentication, rate limiting, request validation, routing.
- **Cache:** Feature cache (Redis), prediction cache (for identical requests), model cache (GPU memory).
- **Load balancer:** Distribute requests across inference replicas.

Orchestration Layer

Coordinates multi-step workflows, agent collaboration, and pipeline execution. Increasingly important in 2026 for agentic AI systems.

- **Agent orchestrator:** Decomposes tasks, delegates to specialized agents, synthesizes results.
- **MCP (Model Context Protocol):** Standardized tool and data source integration for LLMs.
- **Workflow engine:** Airflow, Prefect, Temporal for batch pipelines; LangGraph, CrewAI for agent orchestration.
- **Feedback loop:** Collects user feedback and model outcomes for retraining and drift detection.

Monitoring & Observability Layer (Cross-Cutting)

Spans all layers to provide visibility into system health, model performance, and business impact.

- **Metrics:** Prometheus + Grafana dashboards.

- **Logging:** Structured logs per request for debugging and audit.
- **Tracing:** Distributed tracing (OpenTelemetry, Jaeger) across microservices and inference calls.
- **Drift detection:** Evidently AI, WhyLabs, NannyML — continuous monitoring of data and concept drift.

```
# Layered request flow (recommendation system)
User request → API Gateway → Auth → Feature Fetch (from Feature Store)
→ Cache Check (Redis) → Model Inference → Post-process → Log → Response
```

Exercise: Draw the high-level architecture (on paper or using diagrams.net) for an AI-powered document summarization service. Label each layer with the specific technologies you would use (e.g., Kafka for ingestion, Hugging Face for model, Triton for serving, Prometheus for monitoring).

5. Data Flow

Data flows through an AI system in a continuous loop: ingestion → preprocessing → training → evaluation → deployment → inference → monitoring → feedback.

1. Ingestion

Raw data arrives from multiple sources:

- **Batch sources:** Daily exports from databases, CSV files, S3 buckets.
- **Streaming sources:** Kafka topics, Kinesis streams, event hubs — user clicks, transactions, sensor readings.
- **External APIs:** Weather data, social media feeds, third-party enrichments.

2. Preprocessing

Raw data is cleaned and transformed into features:

```
# Preprocessing pipeline (batch)
Raw data → Validate schema → Handle missing values
→ Normalize/scale → Feature engineering → Feature validation
→ Write to Feature Store (online + offline copies)
```

Key decisions: Do you preprocess in the pipeline (painless) or at serving time (flexible)? The answer depends on latency requirements and feature complexity.

3. Training & Evaluation

Preprocessed features are fed into training jobs. This can happen on a schedule (nightly), on demand (triggered by drift), or continuously (online learning).

- **Training pipeline:** Split data (train/val/test) → train model → evaluate → log metrics → register model if metrics improve.
- **Evaluation:** Compare against champion model. If new model outperforms on key metrics, it moves to staging.

4. Deployment

The trained model is packaged and deployed to the serving infrastructure. Common formats: ONNX, TorchScript, TensorRT, or raw Hugging Face checkpoints.

5. Inference

Live requests flow through the inference pipeline:

```
# Online inference flow
Request → Fetch features (from online feature store + cache)
→ Feature transformation (consistent with training)
→ Model prediction → Post-processing → Return result
→ Log prediction + features to audit store
```

6. Feedback Loop

The most critical and often overlooked step. Model predictions are compared against actual outcomes (once known) and fed back into the system:

- **Explicit feedback:** User ratings, thumbs up/down, corrections.
- **Implicit feedback:** Click-through rates, dwell time, purchase conversion.
- **Outcome labels:** Fraud confirmed/not-confirmed after human review.
- **Drift detection:** Statistical tests comparing prediction distribution to training distribution.

```
# Feedback loop data flow
Inference → Store prediction + features → Wait for ground truth
→ Compute model quality metric (e.g., accuracy, precision)
→ Compare to threshold → If degraded: trigger retraining alert
```

Exercise: Map the complete data flow for a content moderation system that classifies user-uploaded images as safe/unsafe. Identify: a) data sources, b) preprocessing steps, c) where the feedback loop connects, d) what metrics trigger retraining.

6. Key Components

Production AI systems are built from a set of reusable components. Understanding each component's role, technology options, and design considerations is essential.

Feature Store

A centralized repository for features shared across training and serving. Eliminates the training-serving skew caused by inconsistent feature computation.

- **Offline store:** Parquet/AVRO files in S3 for batch training.
- **Online store:** Redis, DynamoDB, Cassandra for low-latency feature lookup during inference.
- **Feature serving API:** gRPC or REST endpoint returning feature vectors for given entity keys.
- **Point-in-time correctness:** Ensures training data uses features as they existed at the prediction time, preventing data leakage.

```
# Feature store usage
# Training: query historical features
features = feature_store.get_historical_features(
    entity_ids=user_ids, feature_names=["avg_purchase_7d", "session_count"],
    timestamp_column="event_time"
)

# Inference: query latest features
features = feature_store.get_online_features(
    entity_ids={"user_id": 42}, feature_names=["avg_purchase_7d"]
)
```

Training Pipeline

Orchestrates data loading, preprocessing, training, evaluation, and model registration.

- **Orchestration:** Airflow, Kubeflow, Prefect, or cloud-native (Vertex AI Pipelines, SageMaker Pipelines).
- **Compute:** GPU clusters (A100, H100), spot instances for cost savings.
- **Artifact storage:** MLflow, W&B, DVC track experiments, metrics, and model binaries.

Model Registry

Versioned catalog of trained models with metadata: metrics, hyperparameters, training dataset ID, deployment status (staging, production, archived).

Serving Infrastructure

Exposes models for inference. Key considerations:

Tool	Best For	Key Feature
NVIDIA Triton	GPU-accelerated serving	Multi-framework, dynamic batching
vLLM	LLM inference	PagedAttention, continuous batching
TorchServe	PyTorch models	Model versioning, A/B testing
BentoML	Python model serving	Unified API, containerization
ONNX Runtime	Cross-framework serving	Optimized execution, quantization

Monitoring Infrastructure

- **Metrics pipeline:** Prometheus collects inference latency, throughput, error rates.
- **Drift detection service:** Runs statistical tests (KS, PSI, chi-squared) comparing training vs serving data.
- **Alerting:** PagerDuty, Slack alerts when metrics breach thresholds.
- **Dashboard:** Grafana dashboards with per-model and per-version views.

Feedback Collection Service

Captures explicit (ratings, corrections) and implicit (clicks, conversions) feedback. Stores it in a feedback data store, timestamped, for future retraining cycles and drift

analysis.

Exercise: List the components you would need for an AI-powered email classification system (spam vs not-spam). For each component, identify: a) what technology you would use, b) what data flows through it, c) what could go wrong if the component fails.